

Energy Prediction

1. Introduction

Energy prediction has become an important research area due to the increasing demand for electricity and the need for efficient energy management. Accurate prediction of electrical power characteristics helps improve resource planning, optimize energy usage, reduce operational costs, and support the development of smart energy systems. Machine learning techniques have proven to be effective in analyzing large volumes of energy data and identifying complex patterns that traditional statistical methods may fail to capture.

In this project, machine learning models were applied to predict household electrical power characteristics, specifically Global Reactive Power, using historical electricity consumption data. The dataset contains electrical measurements collected over time, including active power, reactive power, voltage, current intensity, and sub-metering values. Before building the prediction models, the dataset was cleaned, preprocessed, and analyzed to improve its quality and prepare it for model training.

Two regression algorithms were implemented and compared: Linear Regression and Random Forest Regression. The performance of these models was evaluated using standard regression metrics, including Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and the coefficient of determination (R^2 Score). The comparison was conducted to determine which model provides more accurate predictions of Global Reactive Power based on household electrical measurements.

2. Objectives

The main objective of this project is to develop a machine learning model capable of accurately predicting Global Reactive Power based on historical household electrical measurements. Accurate prediction of reactive power can help improve energy management, optimize resource utilization, and support decision-making in smart energy systems.

The specific objectives of this project are:

- To preprocess and clean the household power consumption dataset by handling missing values and converting data into appropriate numerical formats.
- To perform exploratory data analysis (EDA) in order to understand the characteristics, patterns, and distribution of the electrical measurements.
- To extract useful time-based features from date and time information, such as hour, day, month, year, and weekday, to improve model performance.

- To build and train different machine learning regression models for predicting Global Reactive Power.
- To evaluate the performance of the models using MAE, RMSE, and R² Score.
- To compare the performance of Linear Regression and Random Forest Regression and identify the model that provides the most accurate predictions.

3. Dataset Description

The dataset used in this project is the Household Electric Power Consumption dataset. It contains measurements of electric power consumption collected from a single household over a period of almost four years, from 16 December 2006 to 26 November 2010. The data was recorded at one-minute intervals, making it suitable for time-series analysis and machine learning applications.

The dataset contains approximately 2,049,280 records and several electrical measurements related to household energy consumption. The main features include Global Active Power, Global Reactive Power, Voltage, Global Intensity, and Sub-metering values. These measurements provide detailed information about the electrical behavior of the household and can be used to develop machine learning models for predicting Global Reactive Power.

4. Dataset Features Description

```
Date;Time;Global_active_power;Global_reactive_power;Voltage;Global_intensity;Sub_metering_1;Sub_metering_2;Sub_metering_3
16/12/2006;17:24:00;4.216;0.418;234.840;18.400;0.000;1.000;17.000
16/12/2006;17:25:00;5.360;0.436;233.630;23.000;0.000;1.000;16.000
16/12/2006;17:26:00;5.374;0.498;233.290;23.000;0.000;2.000;17.000
16/12/2006;17:27:00;5.388;0.502;233.740;23.000;0.000;1.000;17.000
16/12/2006;17:28:00;3.666;0.528;235.680;15.800;0.000;1.000;17.000
16/12/2006;17:29:00;3.520;0.522;235.020;15.000;0.000;2.000;17.000
16/12/2006;17:30:00;3.702;0.520;235.090;15.800;0.000;1.000;17.000
16/12/2006;17:31:00;3.700;0.520;235.220;15.800;0.000;1.000;17.000
16/12/2006;17:32:00;3.668;0.510;233.990;15.800;0.000;1.000;17.000
16/12/2006;17:33:00;3.662;0.510;233.860;15.800;0.000;2.000;16.000
16/12/2006;17:34:00;4.448;0.498;232.860;19.600;0.000;1.000;17.000
16/12/2006;17:35:00;5.412;0.470;232.780;23.200;0.000;1.000;17.000
16/12/2006;17:36:00;5.224;0.478;232.990;22.400;0.000;1.000;16.000
16/12/2006;17:37:00;5.268;0.398;232.910;22.600;0.000;2.000;17.000
16/12/2006;17:38:00;4.054;0.422;235.240;17.600;0.000;1.000;17.000
16/12/2006;17:39:00;3.384;0.282;237.140;14.200;0.000;0.000;17.000
16/12/2006;17:40:00;3.270;0.152;236.730;13.800;0.000;0.000;17.000
```

The dataset contains several electrical measurements that describe the electricity consumption behavior of a single household. Each column represents a specific electrical parameter, as described below:

- **Date**

The Date column represents the date when the electrical measurement was recorded. It provides information about the day, month, and year of each observation. This feature was used to extract additional time-based features for improving the machine learning model.

- Time

The Time column represents the exact time of the measurement during the day. It was processed to extract time-related information such as hour and minute, which helps the model capture daily electricity consumption patterns.

- Global Active Power (kW)

The Global Active Power column represents the total active power consumed by the household. Active power is the actual power used to perform useful work, such as running electrical appliances and lighting systems. It is measured in kilowatts (kW).

- Global Reactive Power (kW)

The Global Reactive Power column represents the reactive power consumed by the household. Reactive power is the power required by electrical devices containing inductive components, such as motors and transformers, to create magnetic fields. Although it does not perform useful work, it affects the efficiency and operation of the electrical system. In this project, Global Reactive Power was selected as the target variable to be predicted by the machine learning models.

- Voltage (V)

The Voltage column represents the electrical potential difference supplied to the household. It is measured in volts (V) and indicates the level of electrical pressure available for operating household devices.

- Global Intensity (A)

The Global Intensity column represents the total electric current consumed by the household. It is measured in amperes (A) and indicates the amount of current flowing through the electrical system.

- Sub_metering_1 (Wh)

The Sub_metering_1 column represents the energy consumption measured by the first sub-meter. It indicates the electricity usage of a specific group of household appliances and is measured in watt-hours (Wh).

- Sub_metering_2 (Wh)

The Sub_metering_2 column represents the energy consumption measured by the second sub-meter. It provides information about the electricity consumption of another group of household appliances.

- Sub_metering_3 (Wh)

The Sub_metering_3 column represents the energy consumption measured by the third sub-meter. It indicates the electricity usage of additional household devices and is measured in watt-hours (Wh).

5. Code explanation

Importing Libraries and Loading the Dataset

▼ Step 1: Import Libraries

```
[ ] import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
```

▼ Step 2: Load the Dataset

```
[ ] from google.colab import files
# Upload the dataset from your local computer
files.upload()
# Read the dataset into a Pandas DataFrame
df = pd.read_csv('household_power_consumption.txt', delimiter=';')
# Display the dataset dimensions (rows, columns)
print(df.shape) #2,075,259 (Rows) 9 (Columns)
```

The main Python libraries required for data processing, analysis, and visualization were imported.

- Pandas (pd):
Pandas was used for loading the dataset, handling data structures, and performing data preprocessing operations using DataFrames.
- NumPy (np):
NumPy was imported to perform numerical operations and efficiently handle numerical data.
- Matplotlib:
Matplotlib was used for creating different types of plots and visualizations to analyze the data and identify patterns.
- Seaborn:
Seaborn was used for statistical data visualization and creating more informative graphs to explore relationships and distributions within the dataset.
- The parameter sep=";" was used because the dataset values are separated by semicolons rather than commas.
- The parameter low_memory=False was used to ensure proper data type detection and avoid potential issues when reading large datasets.
- The dataset was stored in a variable called df, which represents the complete dataset in the form of a Pandas DataFrame.

... Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving household_power_consumption.txt to household_power_consumption.txt
(2075259, 9)
/tmp/ipykernel_1469/3065752990.py:5: DtypeWarning: Columns (2,3,4,5,6,7) have mixed types. Specify dtype option on import or set low_memory=False.
df = pd.read_csv('household_power_consumption.txt', delimiter=';')

Exploring the Dataset

Step 3: Explore the Dataset

```
# Display the first 10 rows  
df.head(10)
```

	Date	Time	Global_active_power	Global_reactive_power	Voltage	Global_intensity	Sub_metering_1	Sub_metering_2	Sub_metering_3
0	16/12/2006	17:24:00	4.216	0.418	234.840	18.400	0.000	1.000	17.0
1	16/12/2006	17:25:00	5.360	0.436	233.630	23.000	0.000	1.000	16.0
2	16/12/2006	17:26:00	5.374	0.498	233.290	23.000	0.000	2.000	17.0
3	16/12/2006	17:27:00	5.388	0.502	233.740	23.000	0.000	1.000	17.0
4	16/12/2006	17:28:00	3.666	0.528	235.680	15.800	0.000	1.000	17.0
5	16/12/2006	17:29:00	3.520	0.522	235.020	15.000	0.000	2.000	17.0
6	16/12/2006	17:30:00	3.702	0.520	235.090	15.800	0.000	1.000	17.0
7	16/12/2006	17:31:00	3.700	0.520	235.220	15.800	0.000	1.000	17.0
8	16/12/2006	17:32:00	3.668	0.510	233.990	15.800	0.000	1.000	17.0
9	16/12/2006	17:33:00	3.662	0.510	233.860	15.800	0.000	2.000	16.0

The `df.head()` function was used to display the first five rows of the dataset. This step provides an initial overview of the data and helps verify that the dataset was loaded correctly.

It allows us to check:

- The column names, structure of the dataset and format of the recorded values.

```
# Display information  
df.info()  
  
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 2075259 entries, 0 to 2075258  
Data columns (total 9 columns):  
#   Column              Dtype  
---  ---  
0   Date                object  
1   Time                object  
2   Global_active_power  object  
3   Global_reactive_power object  
4   Voltage             object  
5   Global_intensity     object  
6   Sub_metering_1       object  
7   Sub_metering_2       object  
8   Sub_metering_3       float64  
dtypes: float64(1), object(8)  
memory usage: 142.5+ MB
```

The `df.info()` function was used to obtain general information about the dataset.

It provides:

- The column names , number of non-null values ,the data types of each column and the memory usage of the dataset.

```
# Display statistical summary of numerical columns
df.describe()
```

Sub_metering_3	
count	2.049280e+06
mean	6.458447e+00
std	8.437154e+00
min	0.000000e+00
25%	0.000000e+00
50%	1.000000e+00
75%	1.700000e+01
max	3.100000e+01

The `df.describe()` function provides statistical information about the numerical columns in the dataset, including:

- Count: Number of available values.
- Mean: Average value.
- Standard deviation: Data variation.
- Minimum and maximum values.
- Quartiles (25%, 50%, and 75%).

```
# Display the names of all columns
print(df.columns)
```

```
Index(['Date', 'Time', 'Global_active_power', 'Global_reactive_power',  
      'Voltage', 'Global_intensity', 'Sub_metering_1', 'Sub_metering_2',  
      'Sub_metering_3'],  
      dtype='object')
```

The `df.columns` function was used to display the names of all columns available in the dataset.

Step 4: Check Missing Values

```
# Count missing values in each column
print(df.isnull().sum())
```

```
... Date                0
    Time                0
    Global_active_power  0
    Global_reactive_power  0
    Voltage             0
    Global_intensity     0
    Sub_metering_1       0
    Sub_metering_2       0
    Sub_metering_3      25979
    dtype: int64
```

The `isnull().sum()` function was used to count the number of missing values in each column of the dataset.

The output shows that most columns do not contain any missing values. However, the **Sub_metering_3** column contains **25,979 missing values**, while all other features have zero missing values.

Since missing values can affect the performance of machine learning models, they need to be handled before training the models. Therefore, further preprocessing steps were applied to clean the dataset and prepare it for machine learning algorithms.

Step 5: Data Cleaning

```
# Remove rows containing '?' values in the target-related column
df = df[df['Global_active_power'] != "?"]

# Define columns that should be converted to numerical format
numeric_columns = [
    'Global_active_power',
    'Global_reactive_power',
    'Voltage',
    'Global_intensity',
    'Sub_metering_1',
    'Sub_metering_2',
    'Sub_metering_3']

# Convert columns from object type to numeric type
# Invalid values will be converted into NaN
for col in numeric_columns:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Remove rows containing missing values after conversion
df = df.dropna()

# Create a new feature representing the total sub-metering consumption
df['meter'] = (
    df['Sub_metering_1'] +
    df['Sub_metering_2'] +
    df['Sub_metering_3']
)
```

Removing Invalid Values and Handling Missing Data

- The dataset contains some invalid values represented by the symbol "?". These values cannot be used in machine learning models because they are not numerical values. In this step, rows containing "?" values in the **Global_active_power** column were removed to ensure that the dataset contains valid observations.

Converting Data Types into Numerical Format

The electrical measurement columns were originally stored as object data types because of the presence of non-numerical values.

In this step:

- The selected electrical columns were defined in a list called `numeric_columns`.
- The `pd.to_numeric()` function was used to convert these columns into numerical format.

- The parameter `errors='coerce'` converts any invalid values into NaN (missing values), which allows them to be handled in the next step.

Converting the data into numerical format is necessary because machine learning algorithms require numerical input values.

Removing Missing Values After Conversion

After converting the columns into numerical format, any remaining missing values (NaN) were removed using the `dropna()` function.

Creating a New Feature (Meter)

A new feature called meter was created by combining the three sub-metering measurements.

The new feature represents the total energy consumption measured by the available sub-meters:

`meter = Sub_metering_1 + Sub_metering_2 + Sub_metering_3`

```
# Check the dataset information after cleaning
df.info()

# Check the new dataset size
print(df.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 2049280 entries, 0 to 2075258
Data columns (total 10 columns):
#   Column                Dtype
---  -
0   Date                  object
1   Time                  object
2   Global_active_power    float64
3   Global_reactive_power  float64
4   Voltage                float64
5   Global_intensity       float64
6   Sub_metering_1         float64
7   Sub_metering_2         float64
8   Sub_metering_3         float64
9   meter                  float64
dtypes: float64(8), object(2)
memory usage: 172.0+ MB
(2049280, 10)
```

The output shows that:

- The dataset contains 2,049,280 rows and 10 columns after cleaning.
- All electrical measurement columns were successfully converted to the float64 data type, making them suitable for machine learning algorithms.
- The newly created meter feature was successfully added to the dataset.
- The Date and Time columns remained as object data types because they had not yet been converted into datetime format at this stage.

Step 6: Feature Engineering (Date & Time)

```
#I notice that Date = 16/12/2006 , Time = 17:24:00
# Convert the Date column from string format to datetime format
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)

# Convert the Time column from string format to datetime format
time = pd.to_datetime(df['Time'])
# Extract hour from Time 17:24:00 --> 17
df['hour'] = time.dt.hour
# Extract minute from Time 17:24:00 --> 24
df['minute'] = time.dt.minute
# Extract the day of the week Monday = 0, Sunday = 6
df['weekday'] = df['Date'].dt.weekday
# Extract day of the month 16/12/2006 --> 16
df['day'] = df['Date'].dt.day
# Extract month 16/12/2006 --> 12
df['month'] = df['Date'].dt.month
# Extract year 16/12/2006 --> 2006
df['year'] = df['Date'].dt.year

... /tmp/ipykernel_1469/4079534592.py:8: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consis
time = pd.to_datetime(df['Time'])
```

Feature engineering was performed to extract useful time-based information from the original Date and Time columns. Since machine learning models cannot directly learn from date and time strings, these columns were converted into the datetime format.

After the conversion, several new features were extracted:

- **hour:** Represents the hour of the day (0–23), helping the model capture daily electricity consumption patterns.
- **minute:** Represents the minute of the recorded measurement.
- **weekday:** Represents the day of the week, where Monday is 0 and Sunday is 6. This feature helps distinguish between weekdays and weekends.
- **day:** Represents the day of the month.
- **month:** Represents the month in which the measurement was recorded, allowing the model to capture seasonal variations.
- **year:** Represents the year of the observation, helping the model identify long-term trends in the data.

```
# Display the new features
df[['Date', 'Time', 'hour', 'minute', 'weekday', 'day', 'month', 'year']].head()
```

	Date	Time	hour	minute	weekday	day	month	year
0	2006-12-16	17:24:00	17	24	5	16	12	2006
1	2006-12-16	17:25:00	17	25	5	16	12	2006
2	2006-12-16	17:26:00	17	26	5	16	12	2006
3	2006-12-16	17:27:00	17	27	5	16	12	2006
4	2006-12-16	17:28:00	17	28	5	16	12	2006

The output shows the original Date and Time columns along with the newly extracted features (hour, minute, weekday, day, month and year).

```
df.info()

<class 'pandas.core.frame.DataFrame'>
Index: 2049280 entries, 0 to 2075258
Data columns (total 16 columns):
#   Column                      Dtype
---  -
0   Date                        datetime64[ns]
1   Time                        object
2   Global_active_power         float64
3   Global_reactive_power       float64
4   Voltage                     float64
5   Global_intensity            float64
6   Sub_metering_1              float64
7   Sub_metering_2              float64
8   Sub_metering_3              float64
9   meter                       float64
10  hour                         int32
11  minute                      int32
12  weekday                     int32
13  day                         int32
14  month                      int32
15  year                       int32
dtypes: datetime64[ns](1), float64(8), int32(6), object(1)
memory usage: 218.9+ MB
```

The output confirms that:

- The dataset now contains 16 columns, compared to 10 columns before feature engineering.
- The Date column was successfully converted from object to datetime64[ns].
- Six new time-based features (hour, minute, weekday, day, month, and year) were successfully created and stored as integer (int32) data types.
- All electrical measurement columns remain in float64 format, making them suitable for machine learning algorithms.

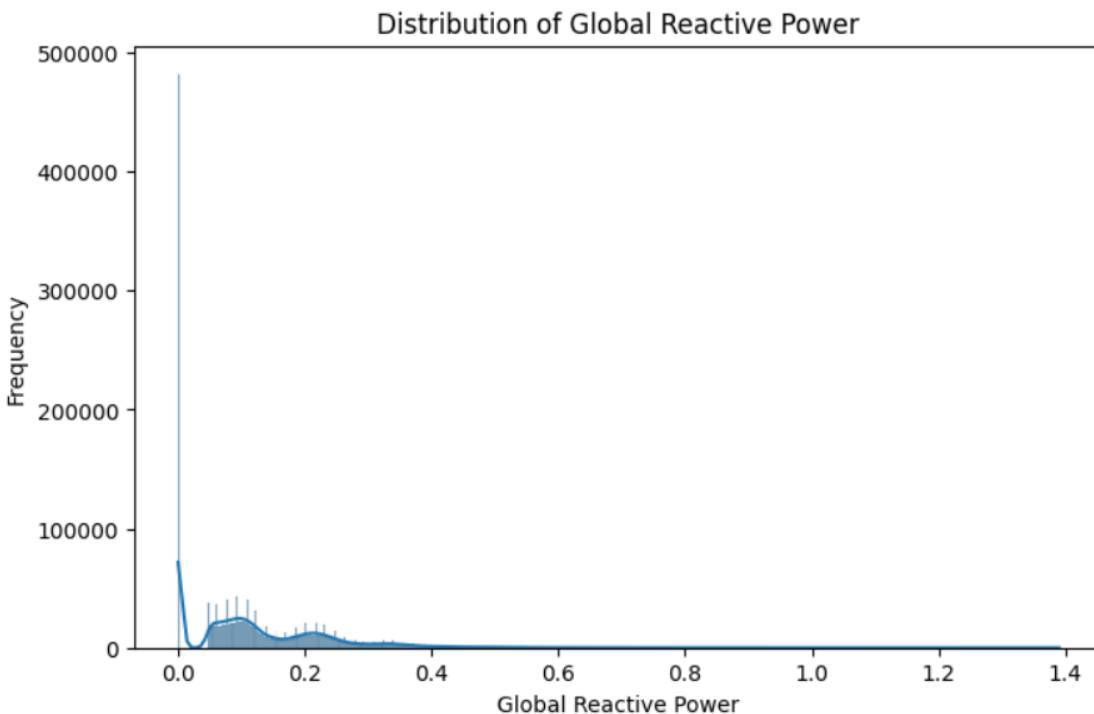
Step 7: Data Visualization

```
#Q1: What is the distribution of Global Reactive Power values?|
plt.figure(figsize=(8,5))

sns.histplot(df['Global_reactive_power'], kde=True)

plt.title("Distribution of Global Reactive Power")
plt.xlabel("Global Reactive Power")
plt.ylabel("Frequency")

plt.show()
```



A histogram with a Kernel Density Estimation (KDE) curve was plotted to examine the distribution of the Global Reactive Power values.

The histogram shows the frequency of observations for different values of Global Reactive Power, while the KDE curve provides a smooth estimate of the overall distribution.

This visualization helps to:

- Understand how the target variable is distributed and identify whether the data is symmetric or skewed.
- Detect the presence of outliers or unusual values and gain insights into the characteristics of the target variable before building machine learning models.

Step 8: Train/Test Split

```
#Define X and y
y=df['Global_reactive_power'] # Target
x=df.drop(columns=["Global_reactive_power", "Date", "Time"])#Features
```

```
x.shape
```

```
(2049280, 13)
```

```
y.shape
```

```
(2049280,)
```

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42 ## Ensures the same split every time we run the code
)
```

Defining Features and Target Variable

Target variable (y): Global_reactive_power, which is the value that the machine learning models are trained to predict.

Input features (X): All remaining numerical variables were used as predictors after excluding the target column and the original Date and Time columns.

The Date and Time columns were removed because their useful information had already been extracted into numerical time-based features (hour, minute, weekday, day, month, and year)

The output indicates that:

- X contains 2,049,280 observations and 13 input features.
- y contains 2,049,280 target values, corresponding to each observation in the dataset.

The matching number of observations in both X and y confirms that the dataset is correctly structured and ready for the next step, which is splitting the data into training and testing sets.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42 ## Ensures the same split every time we run the code
)
```

- x: The input features used for prediction.
- y: The target variable (Global_reactive_power).
- test_size=0.2: Allocates 20% of the dataset for testing and 80% for training.
- random_state=42: Sets a fixed random seed to ensure that the data is split in the same way every time the code is executed, making the results reproducible.

```
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(1639424, 13)
(409856, 13)
(1639424,)
(409856,)
```

The output indicates that:

- X_train contains 1,639,424 observations and 13 input features. This represents 80% of the dataset and is used to train the machine learning models.
- X_test contains 409,856 observations and 13 input features. This represents 20% of the dataset and is used to evaluate the trained models.
- y_train contains 1,639,424 target values corresponding to the training samples.
- y_test contains 409,856 target values corresponding to the testing samples.

The matching number of samples between the feature matrices (**X**) and the target vectors (**y**) confirms that the dataset was split correctly. Therefore, the data is ready for the model training and evaluation stages.

Step 10: Train Models with LinearRegression

```
▶ from sklearn.linear_model import LinearRegression
model = LinearRegression()
# Train the model using the training data
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

The LinearRegression class was imported from the Scikit-learn library and the model was trained using the fit() function, where the training features (X_train) and their corresponding target values (y_train) were provided. During this process, the model learned the relationship between the input features and the target variable.

After training, the predict() function was used to generate predicted values (y_pred) for the testing dataset (X_test).

Step 11: Model Evaluation

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Lower MAE means the predictions are closer to the real values
mae = mean_absolute_error(y_test, y_pred)
print("MAE:", mae)

# It calculates the average of squared differences between actual and predicted values
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print("RMSE:", rmse)

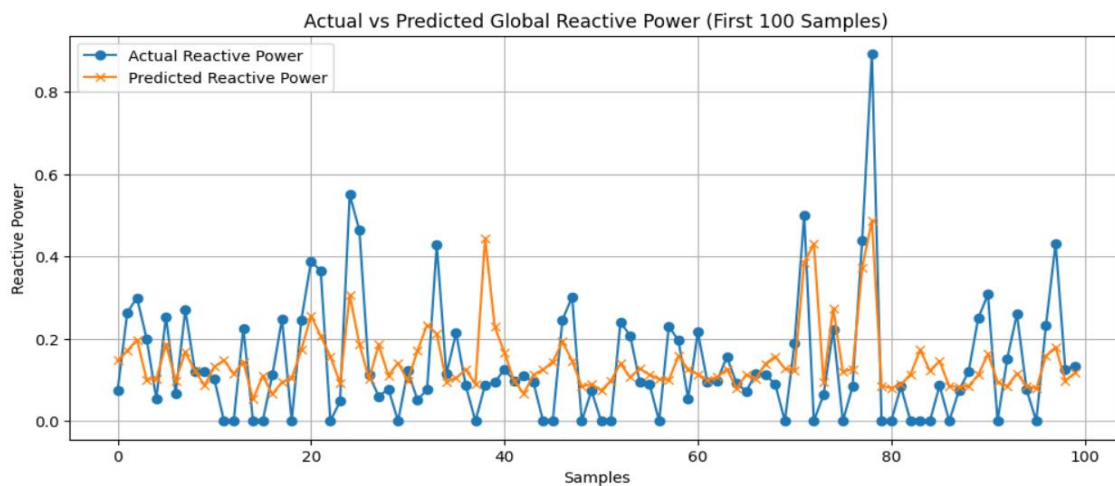
# R2 shows how much variation in the target variable is explained by the model
# Value closer to 1 means better prediction performance
r2 = r2_score(y_test, y_pred)
print("R2 Score:", r2)
```

```
... MAE: 0.07430322862652528
    RMSE: 0.09704504701759126
    R2 Score: 0.2581192106756609
```

- Mean Absolute Error (MAE): Measures the average absolute difference between the predicted values and the actual values. Lower MAE values indicate more accurate predictions.
- Root Mean Square Error (RMSE): Measures the square root of the average squared prediction errors. Since larger errors are penalized more heavily, a lower RMSE indicates better model performance.
- R² Score (Coefficient of Determination): Measures how well the model explains the variation in the target variable. Its value ranges from 0 to 1, where values closer to 1 indicate a better fit.

```
▶ # Create a figure with a specific size
plt.figure(figsize=(12,5))

# Plot the actual values from the test dataset_ real Global_reactive_power values
plt.plot(
    y_test.values[:100],
    label="Actual Reactive Power",
    marker='o')
# Plot the predicted values from the model_predicted by Linear Regression
plt.plot(
    y_pred[:100],
    label="Predicted Reactive Power",
    marker='x')
# Add title to explain the graph
plt.title("Actual vs Predicted Global Reactive Power (First 100 Samples)")
# Label the x-axis
plt.xlabel("Samples")
# Label the y-axis
plt.ylabel("Reactive Power")
# Show legend to distinguish between actual and predicted lines
plt.legend()
# Add grid
plt.grid(True)
# Display the plot
plt.show()
```



Step 11: Train Models with RandomForestRegressor and Evaluation

```
from sklearn.ensemble import RandomForestRegressor

rf_model = RandomForestRegressor(
    n_estimators=100, # Number of decision trees that will be created in the forest
    random_state=42,
    n_jobs=-1        # -1 means use all processors
)

# Train the Random Forest model using training data
rf_model.fit(X_train, y_train)
```

```
... RandomForestRegressor
RandomForestRegressor(n_jobs=-1, random_state=42)
```

The RandomForestRegressor class was imported from the Scikit-learn library.

A Random Forest model was created with the following parameters:

- `n_estimators = 100`: Specifies that the model builds 100 decision trees. The final prediction is obtained by averaging the predictions from all trees, which generally improves accuracy and reduces overfitting.
- `random_state = 42`: Sets a fixed random seed to ensure that the model produces the same results each time the code is executed.
- `n_jobs = -1`: Instructs the algorithm to use all available processor cores, reducing the training time.

Finally, the `fit()` function was used to train the Random Forest model using the training dataset (`X_train` and `y_train`).


```
# Use the trained Random Forest model to predict values for the test data
y_pred_rf = rf_model.predict(X_test)
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae_rf = mean_absolute_error(y_test, y_pred_rf)
print("MAE:", mae_rf)

rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
print("RMSE:", rmse_rf)

r2_rf = r2_score(y_test, y_pred_rf)
print("R2 Score:", r2_rf)
```

```
MAE: 0.020206248877654587
RMSE: 0.04005950999741533
R2 Score: 0.8735849905928049
```

Predicting the Test Data

After training the Random Forest Regression model, the `predict()` function was used to generate predictions for the testing dataset (`X_test`).

- The MAE value of 0.0202 shows that the average prediction error is very small. Similarly, the RMSE value of 0.0401 indicates that the overall prediction errors are low, even when larger errors receive greater emphasis.
- The R^2 Score of 0.8736 means that the model explains approximately 87.36% of the variation in the Global Reactive Power values. This is a significant improvement over the Linear Regression model, demonstrating that Random Forest is much more effective at capturing the complex and nonlinear relationships within the household electricity consumption data.

Overall, the Random Forest Regression model provided more accurate and reliable predictions, making it the preferred model for this energy prediction task.

```

plt.figure(figsize=(12,5))

plt.plot(
    y_test.values[:100],
    label="Actual Reactive Power",
    marker='o'
)

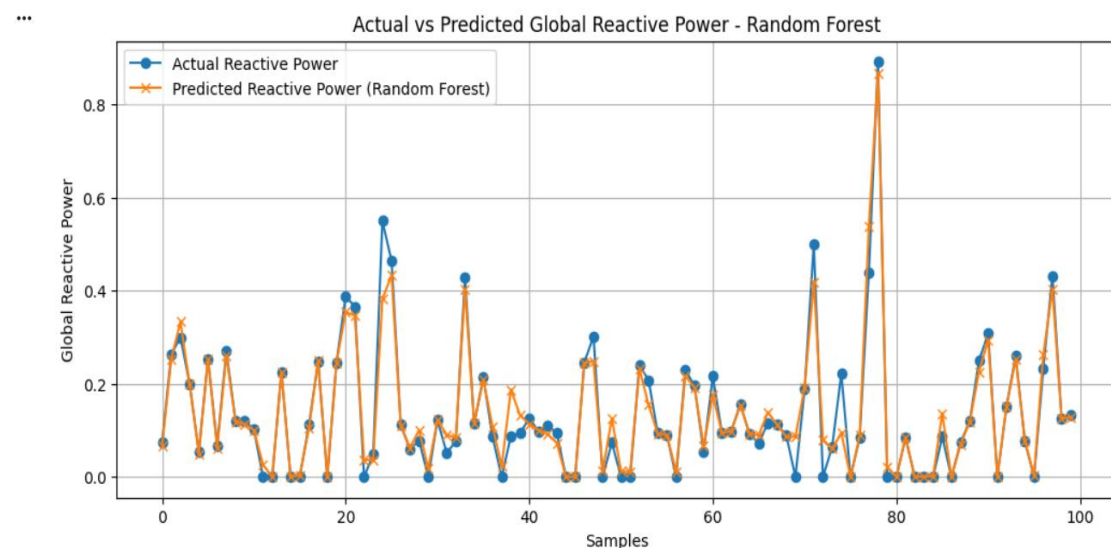
plt.plot(
    y_pred_rf[:100],
    label="Predicted Reactive Power (Random Forest)",
    marker='x'
)

plt.title("Actual vs Predicted Global Reactive Power - Random Forest")
plt.xlabel("Samples")
plt.ylabel("Global Reactive Power")

plt.legend()
plt.grid(True)

plt.show()

```



```

sample = x_test.iloc[[10]]
prediction = rf_model.predict(sample)
print("Predicted:", prediction[0])
# We compare the prediction with the real value to check model accuracy
print("Actual:", y_test.iloc[10])

```

```

Predicted: 0.10079999999999996
Actual: 0.102

```

After evaluating the Random Forest Regression model, a sample from the testing dataset was selected to demonstrate how the trained model makes predictions.

- A single observation was selected from the testing dataset using `X_test.iloc[[10]]`.
- The trained Random Forest model was used to predict the Global Reactive Power value for this sample.

- The predicted value was then compared with the actual value from `y_test`.

This comparison provides a simple example of the model's prediction capability and helps verify that the predicted value is close to the actual value.

6.Conclusion

Comparison Between Linear Regression and Random Forest Regression

<i>Metric</i>	<i>Linear Regression</i>	<i>Random Forest Regression</i>
MAE	0.0743	0.0202
RMSE	0.0970	0.0401
R² Score	0.2581	0.8736

This project focused on predicting Global Reactive Power using household electricity consumption data and comparing the performance of two machine learning models: Linear Regression and Random Forest Regression.

The results show that the Random Forest Regression model significantly outperformed Linear Regression. Random Forest achieved a much lower MAE of 0.0202 and RMSE of 0.0401, compared with 0.0743 and 0.0970 for Linear Regression. In addition, Random Forest achieved an R² score of 0.8736, meaning that it was able to explain approximately 87.36% of the variation in the target variable, while Linear Regression achieved only 25.81%.

The prediction graphs also demonstrate that Random Forest predictions generally follow the actual reactive power values more closely, including several changes and peaks in the data. This indicates that Random Forest is better at capturing the non-linear relationships and complex patterns present in household electricity consumption data.

Overall, the findings demonstrate that Random Forest Regression is a more suitable model for predicting Global Reactive Power in this dataset. The project also highlights the importance of comparing different machine learning algorithms and evaluating them using multiple performance metrics rather than relying on a single measure.

THE END